TWUN13 Cheat Sheet – Last Review Before the Exam

Everything that gives points. **Pass = 30/50** (3: 30–39, 4: 40–45, 5: 46–50). Points: Front end 10 · Browser/server 9 · Code comp 9 · Backend 7 · HTML/CSS/JS 3 · Storing data 3 · Performance 3 · General 2 · Security 2 · State 2.

React – useState & useEffect (Front end 10 + State 2 + most of Code comp 9)

Rule	Remember
Re-render happens when	**state changes** (setter) or **new props**. Normal variables don't re-render.
`useState(0)`	Initial value used **only on 1st render**. State is kept between renders.
Never mutate state	No `count++`, no `user.name = x`. Use setter / new object `{...user, name}`.
`setCount(prev => prev + 1)`	Use when new value depends on old. Safe with batching.
State updates are	**asynchronous and batched** – value is NOT new right after the setter.
setState **in the body**	**Error: Too many re-renders** (infinite loop).
`useEffect(fn)`	After **every** render.
`useEffect(fn, [])`	**Once**, after the 1st render. Logs 0 first, then new value.
`useEffect(fn, [x])`	1st render + when **x changes**. Setting x inside → infinite loop.
`return () => clearTimeout(id)`	**Cleanup**: before next effect + on unmount.
`onClick={count++}`	Wrong. Fix: `onClick={() => setCount(count + 1)}`.
`show && <h1>Hi</h1>`	show true → `<h1>Hi</h1>`. false → nothing. (0 → shows "0")
Fetch data	Inside **useEffect**, never in the body (loop).

React basics	
JSX	HTML syntax in JS → `React.createElement(type, props, children)`. `className`, `htmlFor`, `{value}`.
Functional component	Function that **returns JSX**. PascalCase. Must be pure. Class component = `extends Component`.
Props	Parent → child, **read-only**, an **object**. Make components reusable.
Lists / Fragment	`map` + unique **key**. One root → `<> </>`.
Virtual DOM	New tree → diff with old → apply only changes (batch).
Hooks	useState = state, **useEffect = side effects / fetch**, useContext, useRef, useMemo, useCallback.
Declarative	Say WHAT the UI is, React does the HOW. UI = f(state, props).

Context & Router

Topic	Key points
Context	Solves **prop drilling**. `createContext()` → `<Ctx.Provider value={...}>` (**gives the value**) → `useContext(Ctx)`. Value changes → **all users re-render**. Share state between unrelated components. State is owned by useState, not the context.
Router	**React Router**. `<Link to>`, `<NavLink>` (active), `useNavigate()`, `/products/:id` + `useParams()`, `useSearchParams()` (?sort=), `<Outlet/>` (children), loader + `useLoaderData()`, action + `<Form>`. HashRouter → **#hash** part of URL. **Deep linking** = URL goes straight to content.

JavaScript traps

Code	Result	Why
`console.log(x); var x = 5;`	**undefined**	Hoisting: declaration up, value stays
`console.log(x); let x = 5;`	**ReferenceError**	let/const not usable before the line
`for (var i…) setTimeout(()=>log(i))`	**3 3 3**	var = one shared i (let → 0 1 2)
`5 == "5"` / `5 === "5"`	**true** / false	== converts types, === checks type
`"5" + 1` / `"5" - 1`	**"51"** / 4	+ with string joins text
`"Hello" && 0` / `true && "Hi"`	**0** / "Hi"	&& returns first falsy, else last
Falsy	`false 0 "" null undefined NaN`	Everything else truthy: "0", [], {}
`text[0] = "J"` (text="Hello")	**Hello**	Strings are immutable
`const a=[1]; a.push(2)` / `a = []`	OK / **TypeError**	const locks variable, not content
`obj.missing`	**undefined**	No error for missing property

Code	Result	Why
`if(true){let x=1} log(x)`	**ReferenceError**	let = block scope (var = function scope)
`async f()` called	returns **Promise** right away	`[object Promise]` when printed

Event loop (order of output)

```
1. normal code   2. Promise .then / after await (microtask)   3. setTimeout / setInterval / fs.readFile callback
start, setTimeout(0), Promise.then, end   ->   start, end, promise, timeout
JS is SINGLE-THREADED: events handled one at a time from a FIFO queue (never concurrently).
```

DOM, events, HTML, CSS

- **DOM** = Document Object Model = the API JS uses to read/change the HTML page. `querySelector`, `addEventListener`, `textContent` (safe) vs `innerHTML` (XSS risk).
- Event goes **down** (capture) then **up** (bubble, **default**). `{capture:true}` runs on the way down → body, div, button. `stopPropagation()`, `preventDefault()`. `target` = clicked element, `currentTarget` = element with the listener.
- JS in HTML: attributes (onclick), `<script>` in head, `<script>` anywhere → **all of the above**.
- Link = `<a href>`. `<article>` = self-contained content. XHTML = stricter XML rules (not the same as HTML).
- CSS priority: **id > class > tag**, inline > internal > external, later wins, `!important` wins all. `div p` = all descendants, `div>p` = direct children, `div+p` = next sibling.
- Box model: content → padding → border → margin.

HTTP & browser/server communication (9 + 2 points)

Concept	Answer
Web app	Runs in the browser, talks to a server
Client–server	Client **requests**, server **responds**
HTTP stateless	Each request independent, has all info
Internet vs Web	Web = subset of Internet using **HTTP**
Static site	Fixed content served as-is
URL parts	protocol, host, port, path, query `?`, fragment `#` (**not** file format). Ports 80 / 443
Request	`GET /path HTTP/1.1` + headers + **empty line** (+ body) = the **first 3 lines** are from the browser
Response	`HTTP/1.1 200 OK` + headers + empty line + body (404 can have an HTML page)
API / JSON	How programs talk / text format for data

HTTP methods	Status codes
GET read · POST create · **PUT replace all** · PATCH change part · DELETE · (UPDATE doesn't exist)	2xx ok (200, 201 Created, 204) · 3xx redirect (301 moved, 304) · **4xx client** (400, 401 not logged in, 403 forbidden, **404 not found**) · **5xx server** (500)

fetch & CORS	
fetch	GET by default. `.then(r => r.json())`. **Does NOT reject on 404/500** → check `res.ok`. Only network errors go to `.catch`.
CORS	Lets the browser access **APIs on another domain**. With cookies: server `credentials: true`, fetch `credentials: 'include'`.
Postman	Tool to test HTTP APIs.

Node.js & Express (Backend 7 points)

Thing	Answer
Node.js	**Runtime environment for JS on the server**. No DOM. Not only for HTTP servers.
Express	**Node.js framework** for web apps and APIs
`require('x')`	Imports a module
npm	`npm init` → package.json, `npm install` → node_modules (never push it)
`app.listen(3000)`	**Starts** the server
`app.get(path, handler)`	Route = **method + path**
`app.use(fn)`	**Registers middleware**
Middleware	`(req, res, next)`. Runs **in order**. `next()` → next one. Can **end the cycle** (res.send). Can change req/res. All methods.
req.query / req.params / req.body	`?age=25` / `/users/:id` (string!) / POST body (needs `express.json()`)

Thing	Answer
res.send / json / status / end / set	body / JSON / status code / empty / **set header**
`res.status(404).send('Not Found')`	Sends status 404 **and** the text to the client
No route matches	"Cannot GET /path"
`console.log` at file bottom	Runs right away at startup, before any request
EventEmitter	Calls listeners **synchronously in registration order**

Storing data & auth (3 points + backend)

Thing	Answer
MongoDB	**NoSQL document database** (JSON/BSON)
Structure	database → **collection** (holds documents) → document
`_id`	**Uniquely identifies** each document
NoSQL advantage	**Scalability and flexibility** (horizontal scaling, no fixed schema, no joins)
Schema design (Mongo)	Shape data after **query patterns**
SQL	Tables, fixed schema, joins, **ACID** (Atomicity, Consistency, Isolation, Durability), vertical scaling
NoSQL types	key-value (Redis), column (Cassandra), graph (Neo4j), document (MongoDB)
Mongoose / model / ORM	Schema-based MongoDB for Node / constructor compiled from a Schema / objects ↔ tables
Authentication / authorization	Who are you? / What may you do?
Session	Data on the **server**; browser keeps only the **session ID** in a cookie
Cookie	**Small text data** from the server, stored in the browser, sent automatically. Client and server can read/change. Not encrypted. No passwords!
Cookie flags	HttpOnly (no JS), Secure (HTTPS only), SameSite, Expires
Passwords	**Hash + salt**, never plain text. Wrong login → **401**

Security & performance (2 + 3 = 5 easy points)

STRIDE	
Spoofing	Pretend to be someone else (stolen login)
Tampering	Change data/code (script changes homepage)
Repudiation	Deny an action – system **can't trace/log** it
Information disclosure	Data leaks (sent **unencrypted**, read forbidden file)
Denial of Service	Make service **unavailable** (flood with traffic)
Elevation of privilege	Get more rights (user → admin)

Term	Answer
SQL injection	"Bobby Tables" – input becomes SQL. Fix: sanitize input
XSS / CSRF	Evil JS injected in a page / user tricked into sending a request
HTTPS	Encrypts → protects against **man-in-the-middle**
CDN	Global servers, content from the **nearest** one (cached copies)
Caching	**Store temporary data** to improve performance
Load balancing	**Spread traffic** over many servers
Lazy loading	Load images/scripts **only when needed**
Load testing	Performance under **expected user load**
TypeScript `any`	Can hold **any type** (no type checking)

> **Remember:** T10 answer recipe: **output** + **concept name** + **one sentence on why**. Max ~50 words.